算法作业六

2025 年 10 月 27 日

调研学习线性规划的常见算法

1 问题分析（学习笔记）

线性规划

1.模型构建

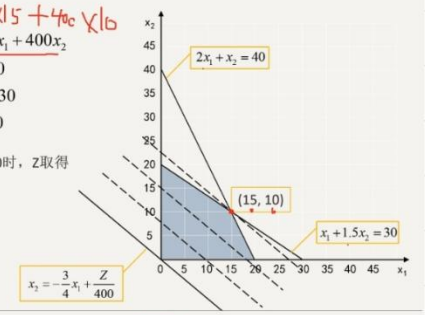
2.图解法

图解法

$\max Z = 300x_1 + 400x_2$

$\begin{cases} 2x_1 + x_2 \le 40 \\ x_1 + 1.5x_2 \le 30 \\ x_1 \ge 0; x_2 \ge 0 \end{cases}$

当x_1=15，x_2=10时，Z取得最大值8500

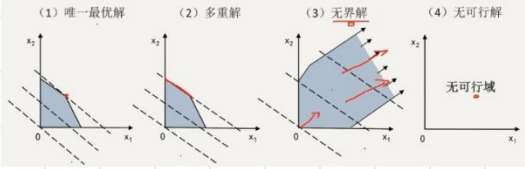


图解法

可行解：满足线性规划模型约束条件的解

最优解：使目标函数达到最大（小）值的可行解

线性规划的解有四种情况：



3.标准型

标准型

线性规划模型的标准型：

两种形式

$$\max Z = \sum_{j=1}^{n} c_j x_j$$

$$\begin{cases} \sum_{j=1}^{n} a_{ij} x_j = b_i, i = 1,2,\dots,m \\ x_j \ge 0, j = 1,2,\dots,n \end{cases}$$

矩阵形式

$$\max Z = CX$$

$$\begin{cases} AX = b \\ X \ge 0 \end{cases}$$

标准型要求

（1）目标函数为求最大值

（2）约束条件均为等式方程

（3）变量x_j为非负

（4）常数b_i都大于等于零

目标：

① 目标函数为max的形式

② 多变量的式子都是等式

③ 右侧数字是≥0的

④ 式子中的变量都≥0

解题方法：

① 将$\min \alpha = a$ 变成$\max \beta = -a$

② 将大括号中多变量的式子都变成等式

（给不等式加上或减去某个≥0的x_n，就能变成等式）

③ 使所有式子最右边的数都≥0

④ 将大括号中单变量的式子都变成啥≥0的形式

若某变量$x_m \le 0$，则设一个≥0的新变量x_m'

使$x_m' = -x_m$，这样可以将$x_m \le 0$变成$x_m' \ge 0$

⑤ 若某变量x_p没提到\是自由变量\是无约束变量

这说明x_p可正可负可0，则设两个≥0新变量x_p'、x_p''

使$x_p' - x_p'' = x_p$

（因为俩≥0的数相减，结果就是可正可负可0）

（比如，5−3结果为正，3−5结果为负，3−3结果为0）

⑥ 把大括号里所有单变量的式子，写到最下面一行

1、变量x_3无符号约束：

令$x_3 = x_3' - x_3''$ （$x_3' \ge 0$，$x_3'' \ge 0$）

2、第一个约束条件是“≤”号：

在不等式左边加入松弛变量x_4($x_4 \ge 0$)化为等式：

$2x_1 + x_2 + x_3' - x_3'' + x_4 = 8$

3、第二个约束条件是“≥”号：

在不等式左边减去剩余变量(或称松弛变量)x_5($x_5 \ge 0$)化为等式：

单纯形法

① 将题目给出的线性规划问题化为标准型
目标函数和大括号中的每个多变量的式子中都要包含所有变量

② 根据标准型填写初始单纯形表

c_j							
c_B	X_B	x_1	x_2	…	x_n	b	θ_i
σ_j							

1. 列数等于变量的个数
2. 行数等于大括号中多变量的式子的个数
3. c_j行的数字根据目标函数中各变量的系数找
4. 每个变量所在列的数字根据大括号中多变量式子中该变量的系数找
5. 找出变量下方已填写数字构成的矩阵中的单位矩阵，依次将该单位矩阵对应的变量写在X_B的下面
6. 将5中找到的变量上方的数字依次写在c_B的下面
7. 将大括号中多变量式子右侧数字依次填到b下面

③ 找出可行解
令X_B所在列的变量与b所在列的数字对应相等，再令其他变量等于0

④ 求出检验数 $\sigma_j = c_j - (c_{B1} \cdot x_{j1} + c_{B2} \cdot x_{j2} + \cdots)$
X_B下面的变量对应的σ的值为0
初始单纯形表中c_j行的数字与σ_j行的数字完全相等

⑤ 观察一下 σ_j 这一行的数字看一下是否都≤0
若这些数字都≤0，则该可行解就是最优解
若这些数字有＞0的，则该可行解不是最优解继续进行⑥

⑥ 找到σ_j 行最大的数字那一列对应的变量x_a(进基变量)
求出 $\theta_i = b_i \div x_{a_i}$
【若θ≥0，则把θ的值填到表中
若θ＜0，则不用把θ的值填到表中
若θ无意义，则不用把θ填到表中】
【注意：若求出来的θ都＜0，则该线性规划问题的解为无界解】

⑦ 找到表中 θ_i 最小值对应 X_B 列的变量 x_b (出基变量)
找到x_a的列和 x_b的行交叉的数字m

⑧ 用x_a上面的数字替代x_b前面的数字，用x_a替代x_b，清空σ_j 行与θ_i列

⑨ 对 x_1、 $x_2 \cdots x_n$ 与 b 列组成的矩阵进行运算，将m变成1，同列其他元素变成0，形成一个新的矩阵，将该矩阵中的数字填入表格中对应的位置形成新的单纯形表并进行步骤③

将m所在行的所有元素，都除以m

把m所在行的各元素乘以同一个数然后加到要变的元素所在行对应的元素上去

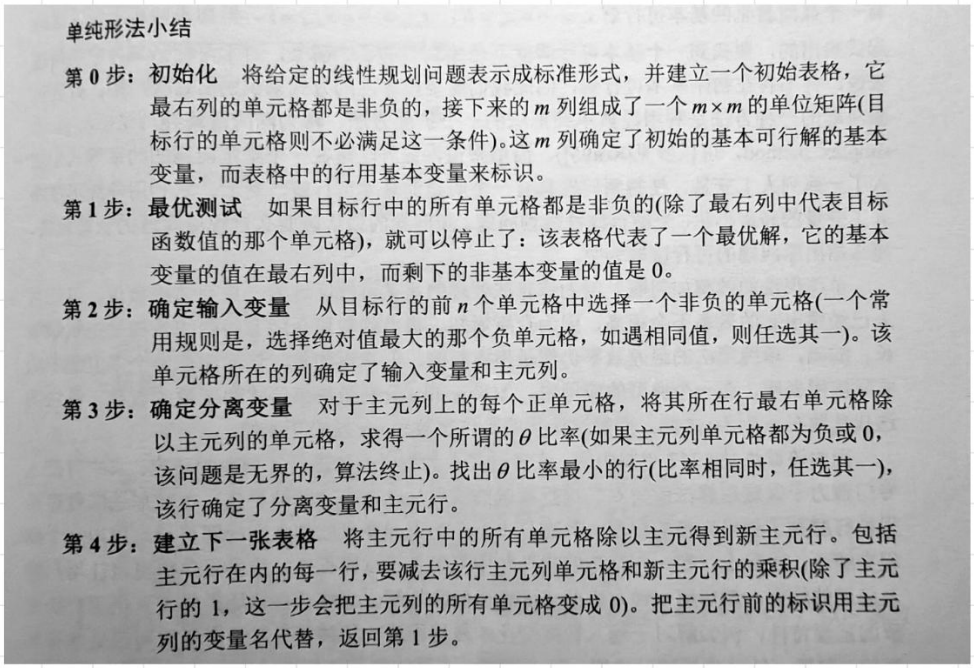
如何检验所求的最优解是不是唯一最优解：

a. 找出X_B列变量以外的变量

b. 找出X_B列变量以外的变量对应的 σ_j

c. 若X_B列变量以外的变量对应的 σ_j都＜0，则X^* 为唯一最优解

若X_B列变量以外的变量对应的 σ_j中有＝0的，则 X^*是无穷多最优解中的一个



单纯形法小结

第 0 步：初始化 将给定的线性规划问题表示成标准形式，并建立一个初始表格，它最右列的单元格都是非负的，接下来的 m 列组成了一个 $m \times m$ 的单位矩阵(目标行的单元格则不必满足这一条件)。这 m 列确定了初始的基本可行解的基本变量，而表格中的行用基本变量来标识。

第 1 步：最优测试 如果目标行中的所有单元格都是非负的(除了最右列中代表目标函数值的那个单元格)，就可以停止了：该表格代表了一个最优解，它的基本变量的值在最右列中，而剩下的非基本变量的值是 0。

第 2 步：确定输入变量 从目标行的前 n 个单元格中选择一个非负的单元格(一个常用规则是，选择绝对值最大的那个负单元格，如遇相同值，则任选其一)。该单元格所在的列确定了输入变量和主元列。

第 3 步：确定分离变量 对于主元列上的每个正单元格，将其所在行最右单元格除以主元列的单元格，求得一个所谓的 θ 比率(如果主元列单元格都为负或 0，该问题是无界的，算法终止)。找出 θ 比率最小的行(比率相同时，任选其一)，该行确定了分离变量和主元行。

第 4 步：建立下一张表格 将主元行中的所有单元格除以主元得到新主元行。包括主元行在内的每一行，要减去该行主元列单元格和新主元行的乘积(除了主元行的 1，这一步会把主元列的所有单元格变成 0)。把主元行前的标识用主元列的变量名代替，返回第 1 步。

2 算法思路

直接用二维数组模拟单纯形表（tableau）：

设：

- m：约束数；
- n：变量数；
- 表格大小为 (m+1) × (n+m+1)；
  - 前 n 列是原变量；
  - 接下来的 m 列是松弛变量；
  - 最后一列是 b；
- 最后一行是 σ（目标函数）。

3 C 语言实现代码 还不太会写（bushi

```c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

#define MAX_M 10
#define MAX_N 10
#define EPS 1e-9

void print_table(double T[MAX_M + 1][MAX_N + MAX_M + 2], int m, int n) {
    printf("\nTableau:\n");
    for (int i = 0; i <= m; i++) {
        for (int j = 0; j <= n + m; j++) printf("%8.3f ", T[i][j]);
        printf("\n");
    }
}

int main() {
    int m = 3, n = 2; // 例子: max 3x1+2x2
    double A[3][2] = { {1,1},{1,0},{0,1} };
    double b[3] = { 4,2,3 };
    double c[2] = { 3,2 };

    double T[MAX_M + 1][MAX_N + MAX_M + 2] = { 0 };
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) T[i][j] = A[i][j];
        T[i][n + i] = 1; // slack
        T[i][n + m] = b[i];
    }
    // 目标行
    for (int j = 0; j < n; j++) T[m][j] = -c[j];

    print_table(T, m, n);

    while (1) {
        // 找入基变量
        int pivot_col = -1;
        double max_sigma = 0;
        for (int j = 0; j < n + m; j++) {
            if (T[m][j] > max_sigma + EPS) {
                max_sigma = T[m][j];
```

```
            pivot_col = j;
        }
    }
    if (pivot_col == -1) break; // 最优

    // 比值检验
    int pivot_row = -1;
    double min_theta = 1e9;
    for (int i = 0; i < m; i++) {
        if (T[i][pivot_col] > EPS) {
            double theta = T[i][n + m] / T[i][pivot_col];
            if (theta < min_theta - EPS) {
                min_theta = theta;
                pivot_row = i;
            }
        }
    }
    if (pivot_row == -1) {
        printf("无界解! \n");
        return 0;
    }

    // 枢轴
    double pivot = T[pivot_row][pivot_col];
    for (int j = 0; j <= n + m; j++) T[pivot_row][j] /= pivot;

    for (int i = 0; i <= m; i++) {
        if (i == pivot_row) continue;
        double factor = T[i][pivot_col];
        for (int j = 0; j <= n + m; j++)
            T[i][j] -= factor * T[pivot_row][j];
    }

    print_table(T, m, n);
}

printf("\n最优解: \n");
printf("Z = %.3f\n", T[m][n + m]);
return 0;
}
```

4 运行示例

```
Tableau:
   1.000    1.000    1.000    0.000    0.000    4.000
   1.000    0.000    0.000    1.000    0.000    2.000
   0.000    1.000    0.000    0.000    1.000    3.000
  -3.000   -2.000    0.000    0.000    0.000    0.000

最优解:
Z = 0.000
```

5 性能分析

项目	描述
时间复杂度	单次迭代更新表格约 $O(mn)$，迭代次数在理论最坏情况下为指数级，但在实际问题中接近多项式性能。
空间复杂度	主要为存储表格，空间 $O(m(n+m))$。